



BSc (Hons) in **Information Technology**
Specialized in AI

IT3012 : Intelligent Agents

Year 3 · Semester 1 · 2026 Semester 2

Faculty of Computing

Practical 03

Objective & Lecture Mapping: In previous labs, we built a Simple Reflex Agent that moved randomly or reacted only to immediate adjacent cells. Today, we transition to a **Goal-Based/Planning Agent**. You will expose the environment's state space to the agent, allowing it to use **Breadth-First Search (BFS)**, **Depth-First Search (DFS)**, and **Uniform-Cost Search (UCS)** to simulate and find paths to the food before taking a physical action.

Part 1: Practical Implementation — Building the Search Agent

Step 1.1: Exposing the World Model

Classical search algorithms require an abstract model of the environment to simulate future states. Currently, your agent is "blind" to the overall map.

1. Create a new Branch called Week_03 in your Forked Github Repo and work on the below Questions.
2. Open `visual_grid_game.py` .
3. Locate the `get_percept(self)` method.
4. Add three new keys to the dictionary to pass the global state to the agent:

```
'grid_size': (self.width, self.height)
'walls': list(self.walls)
'all_food': list(self.food_positions)
```

Step 1.2: Implementing BFS, DFS, and UCS

You will implement three distinct uninformed search strategies. The core node expansion structure is identical; only the data structure for the **Frontier** changes!

1. Open `agent.py` and create a new class called `SearchAgent`.
2. Import required structures: `from collections import deque` and `import heapq`.
3. **BFS:** Create `def bfs_search(...)`. Use a FIFO queue (`deque.popleft()`). This explores the shallowest nodes first.
4. **DFS:** Create `def dfs_search(...)`. Use a LIFO stack (`list.pop()`). This explores the deepest nodes first.
5. **UCS:** Create `def ucs_search(...)`. Use a Priority Queue (`heapq.heappop()`) ordered by the total path cost $g(n)$.
6. For all three algorithms, you must maintain a `reached` set (or visited array) to track explored states. This converts your algorithm from a *Tree Search* to a *Graph Search*, preventing infinite loops.

Step 1.3: Executing and Comparing Offline Plans

The agent must now form a complete plan and execute it step-by-step.

1. In your `SearchAgent.__init__`, add an empty list: `self.plan = []` and a config string `self.active_algo = 'BFS'`.
2. In `sense_and_act(self, percept)`, check if `self.plan` is empty.
3. If empty, find the closest food pellet from `percept['all_food']`, execute the search method matching `self.active_algo`, and store the resulting sequence of actions in `self.plan`.
4. Return the first action from the plan using `return self.plan.pop(0)`.
5. **Observation Task:** Change `self.active_algo` between 'BFS', 'DFS', and 'UCS' and run the simulation. Watch how DFS takes erratic, winding paths, while BFS and UCS take direct, optimal paths!

Part 2: Theoretical Evaluation

Based on your implementation and Lecture 04, complete the following analytical questions.

1. (Remember) You built your search logic based on the environment coordinates. What is the fundamental difference between the "State Space" and the "Search Tree"?

In my agent.py the state space is basically just every (x,y) cell on the grid that isn't a wall - its fixed by grid_size and self.walls from get_percept(). The search tree is different, its built while the algorithm is running. Each thing i push into the queue/stack like (current, path) is actually a whole path from start to that cell, so the same cell can show up more than once in the tree if there's more than one way to get there.

2. (Understand) In Step 1.2, you were instructed to use a `reached` set. In graph search theory, what specific problem does this set solve, and what would happen to your DFS agent without it?

The reached set (reached = {start}) is there so i dont add the same neighbor into the frontier twice - i check "if neighbor not in reached" before pushing it. This solves the repeated state problem basically. If i didnt have this in dfs_search, since stack.pop() just keeps diving deeper and the grid has open spaces where you can walk in a loop, dfs would just keep pushing the same cells over and over and probably never stop / take forever before it finds the food.

3. (Analyze) Compare the visual paths generated by BFS and DFS in your simulation. Why is BFS guaranteed to be optimal in this specific grid, whereas DFS produces winding, suboptimal routes?

In get_neighbors every move only costs 1 (Up/Down/Left/Right are all equal), so shortest path = cheapest path here. bfs_search uses queue.popleft() which is FIFO so it explores level by level, meaning it checks everything 1 step away before anything 2 steps away etc. so the first time it reaches goal thats guaranteed the shortest path. dfs_search uses stack.pop() (LIFO) so it just commits to one direction and goes as deep as possible before backtracking, it stops whenever it happens to bump into the goal not because it found the best route, thats why the path looks messy/winding.

4. (Evaluate) If we scaled `visual\_grid\_game.py` up to a massive 1000x1000 grid, BFS and UCS might crash before finding food. According to the time and space complexity formulas from the lecture, contrast the memory bottlenecks of BFS versus DFS.

1. On something like a 1000x1000 grid, bfs_search and ucs_search would probably run out of memory before running out of time. that's because both of them keep the whole frontier (queue / priority_queue) in memory at once, plus each entry also stores its full path list, and the reached set/dict keeps growing too -.could end up with hundreds of thousands of entries. dfs_search on the other hand only keeps one path on the stack at a time really (plus leftover neighbors), so its memory usage is way smaller, more linear. so dfs won't crash from memory as easily but it might return a really long, bad path since it doesn't care about finding the shortest one.



Once Completed Get a PDF of the completed File and Push into the Week 03 branch in the Github Project

Finalize and Lock Lab 03 Documentation

Incomplete: 0/11 questions answered. Please provide detailed responses for all questions.



FACULTY OF COMPUTING